



**Faculty of Engineering and Technology
Electrical and Computer Engineering Department**

Operating Systems ENCS3390

First Semester 2024 / 2025

First Project

Student Name: Veronica Wakileh

University ID: 1220245

Instructor's name: Dr. Abdel Salam Sayyad

Section: 2

Date: 8 /12 /2024

Abstract

This project will discuss the efficiency of three computational approaches including naive single-threaded approach, multiprocessing, and multithreading that analyze a text dataset for the top 10 most frequent words. Experiments were conducted on a system with at least four cores to evaluate performance with different numbers of child processes and threads, such as 2, 4, 6, and 8. Amdahl's law was used to determine the serial and parallel proportion of code, studying the theoretical and practical scalability of each approach. Execution times were measured and compared in order to find the best configuration for each method. The analysis gives some insight into the practical use of multiprocessing and multithreading in computational tasks, pointing out their importance in performance optimization for data-intensive operations.

Table of contents

Abstract.....	II
Table of contents	III
Table of Figures.....	III
List of Tables:	V
Theory:.....	1
1. Naive Approach:	1
2. Multiprocessing Approach:.....	1
3. Multithreading Approach:	1
Results:.....	2
1. Naive Approach:	2
2. Multiprocessing Approach:.....	3
3. Multithreading Approach:	5
1- Describe your environment: computer specs (Cores? Speed? Memory?), OS, programming language, and IDE tool, and whether you used a virtual machine.	7
2- How you achieved the multiprocessing and multithreading requirements, i.e. The API and functions that you used.	9
Multiprocessing (Using fork () and wait ())	9
Multithreading (Using POSIX Threads).....	9
3- An analysis according to Amdahl's law. What percentage is the serial part of your code? What is the maximum speedup according to the available number of cores? What is the optimal number of child processes or threads?	10
What is the optimal number of child processes or threads?	10
4- A table that compares the performance of the 3 approaches.	11
5- Comment on the differences in performance	12
Conclusion:	13

Table of Figures

Figure 1: Naive approach result	2
Figure 2: Multiprocessing result (8 child processes)	3
Figure 3: Multiprocessing result (6 child processes)	3
Figure 4: Multiprocessing result (4 child processes)	4
Figure 5: Multiprocessing result (2 child processes)	4
Figure 6: Multithreading result (8 child threads)	5
Figure 7 : Multithreading result (6 child threads)	5
Figure 8: Multithreading result (4 child threads)	6
Figure 9: Multithreading result (2 child threads)	6
Figure 10: Operating System information	7
Figure 11: The Device's and window's OS specifications	8
Figure 12: Processor's related details	8
Figure 13: Parallel execution time code	10

List of Tables:

Table 1: Comparison of the execution times of the three approaches.....	10
--	-----------

Theory:

1. Naive Approach:

The naive approach executes the tasks sequentially one after the other, within a single process. That is, if one task takes a long time to complete, it can delay the execution of subsequent tasks. Since there's only one process, there is no parallelism, CPU resources are not utilized effectively.

2. Multiprocessing Approach:

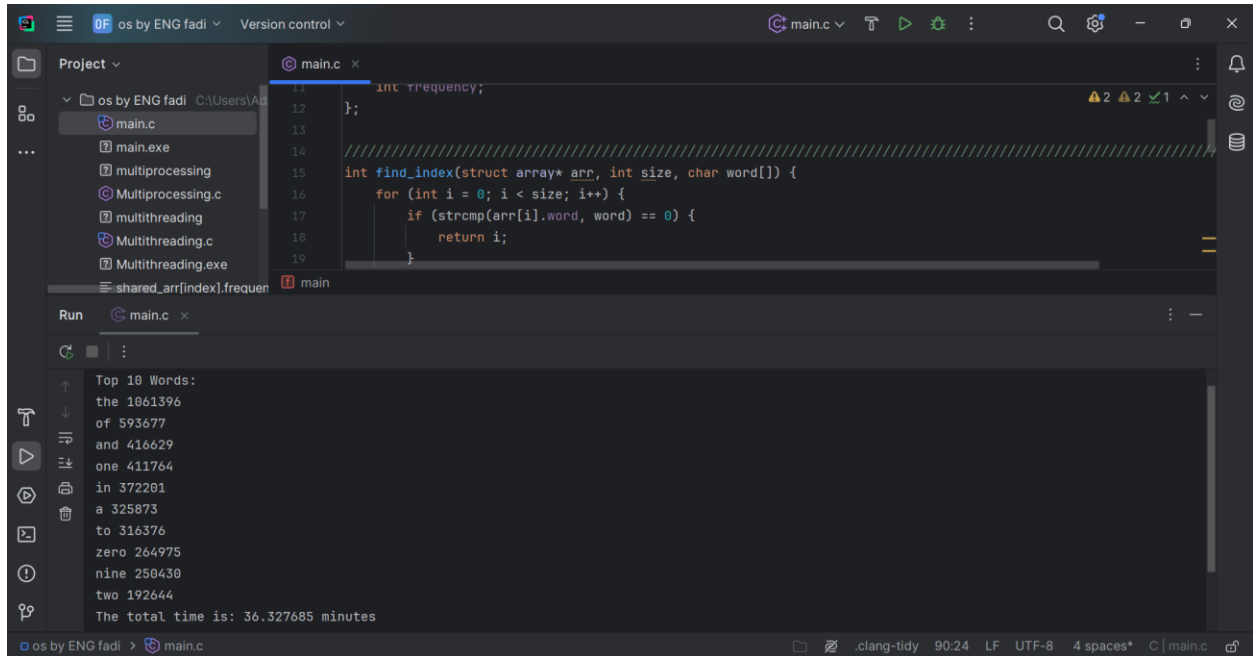
In multiprocessing, several child processes are spawned, each running separately from the others. This allows jobs to be executed simultaneously, taking advantage of multiple CPU cores if available. By distributing the workload across multiple processes, the overall performance can be significantly improved compared to the naive approach. However, creating and managing processes can incur some overhead.

3. Multithreading Approach:

Multiple threads of a same process are created and all threads execute different tasks of a program simultaneously. When different threads are executing their respective tasks, as threads shares the memory space, then obviously the communication between the threads is faster compared to processes. In tasks involving I/O operations or can be parallelized effectively, multithreading works as an advantageous way. Though it brings complexity in managing threads and problems like race condition or deadlocks may arise in managing shared resources.

Results:

1. Naive Approach:



```
11 int frequency;  
12 };  
13  
14 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////  
15 int find_index(struct array* arr, int size, char word[]) {  
16     for (int i = 0; i < size; i++) {  
17         if (strcmp(arr[i].word, word) == 0) {  
18             return i;  
19         }  
20     }  
21 }
```

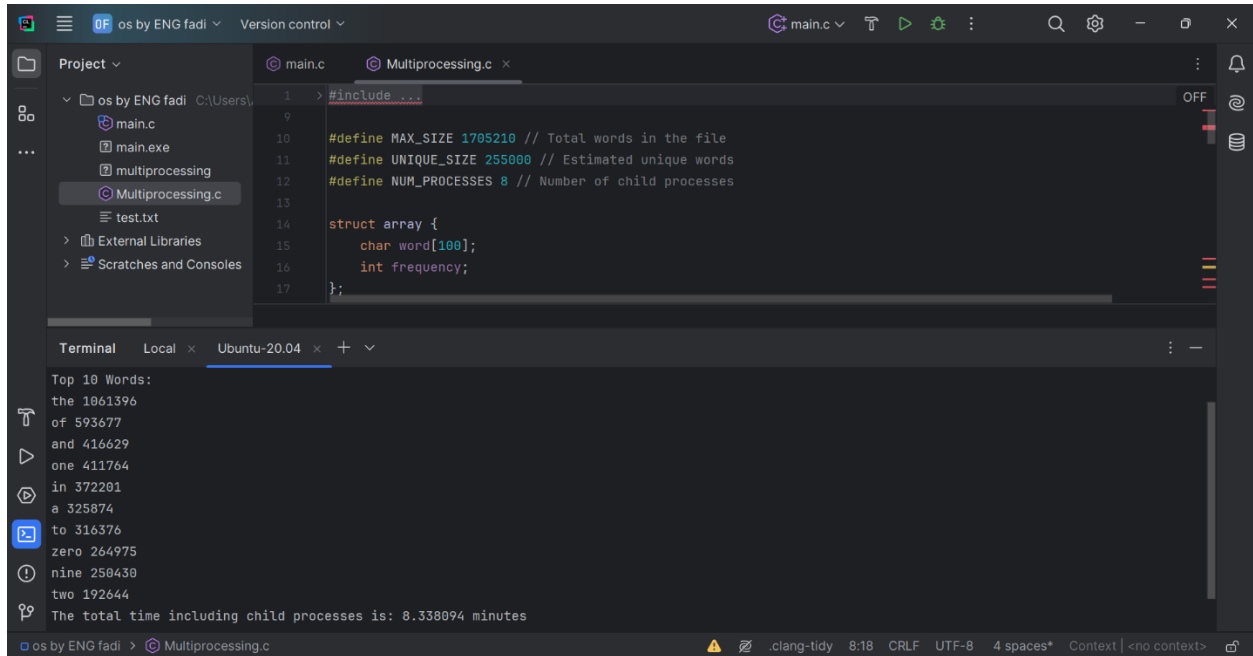
Run main.c x

Top 10 Words:
the 1061396
of 593677
and 416629
one 411764
in 372201
a 325873
to 316376
zero 264975
nine 250430
two 192644
The total time is: 36.327685 minutes

Figure 1: Naive approach result

I tried the same code earlier on another PC but it took less time due to better system's specifications and then speed up would be higher.

2. Multiprocessing Approach:



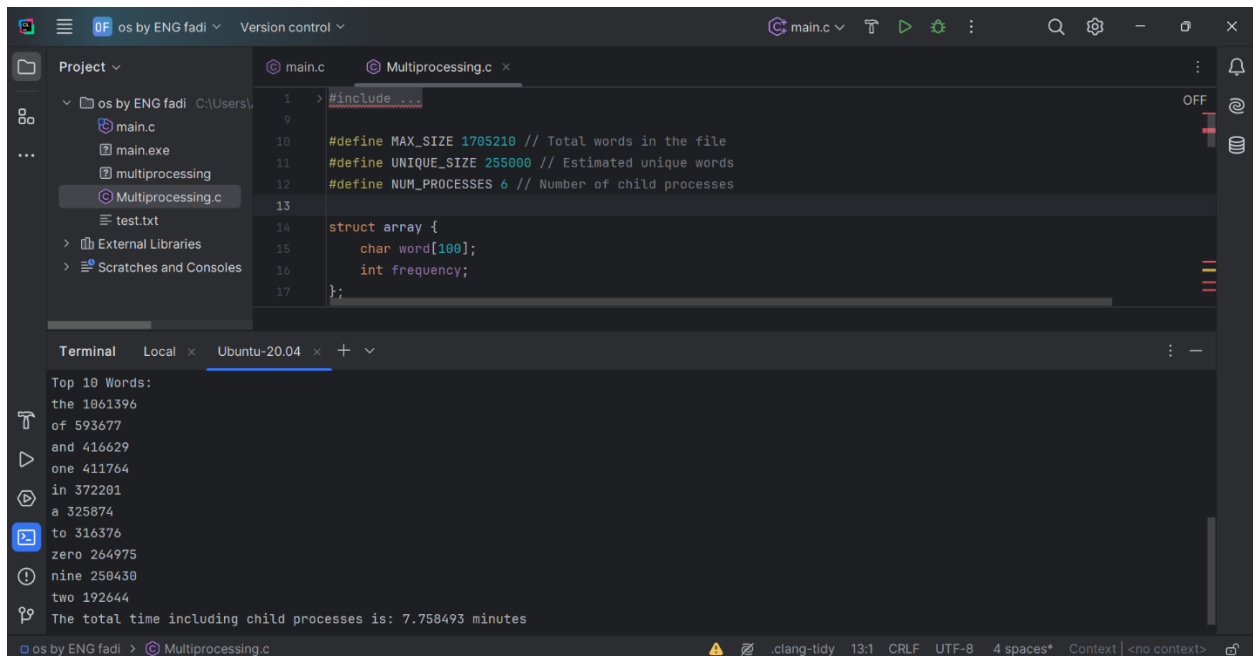
The screenshot shows the Visual Studio Code interface with the 'Multiprocessing.c' file open. The code defines 8 child processes. The terminal output shows the top 10 words and the total time including child processes is 8.338094 minutes.

```
1 > #include <stdio.h>
2
3 #include <stdlib.h>
4 #include <string.h>
5 #include <unistd.h>
6 #include <sys/types.h>
7 #include <sys/wait.h>
8 #include <pthread.h>
9
10 #define MAX_SIZE 1705210 // Total words in the file
11 #define UNIQUE_SIZE 255000 // Estimated unique words
12 #define NUM_PROCESSES 8 // Number of child processes
13
14 struct array {
15     char word[100];
16     int frequency;
17 };
18
19 void *word_count(void *arg) {
20     struct array *arr = (struct array *) arg;
21     FILE *f = fopen("test.txt", "r");
22     if (f == NULL) {
23         perror("fopen");
24         return NULL;
25     }
26     while (!feof(f)) {
27         char word[100];
28         fscanf(f, "%s", word);
29         if (word[0] != '\0') {
30             arr->frequency++;
31         }
32     }
33     fclose(f);
34     return arr;
35 }
```

Terminal Output:

```
Top 10 Words:
the 1861396
of 593677
and 416629
one 411764
in 372201
a 325874
to 316376
zero 264975
nine 258430
two 192644
The total time including child processes is: 8.338094 minutes
```

Figure 2: Multiprocessing result (8 child processes)



The screenshot shows the Visual Studio Code interface with the 'Multiprocessing.c' file open. The code defines 6 child processes. The terminal output shows the top 10 words and the total time including child processes is 7.758493 minutes.

```
1 > #include <stdio.h>
2
3 #include <stdlib.h>
4 #include <string.h>
5 #include <unistd.h>
6 #include <sys/types.h>
7 #include <sys/wait.h>
8 #include <pthread.h>
9
10 #define MAX_SIZE 1705210 // Total words in the file
11 #define UNIQUE_SIZE 255000 // Estimated unique words
12 #define NUM_PROCESSES 6 // Number of child processes
13
14 struct array {
15     char word[100];
16     int frequency;
17 };
18
19 void *word_count(void *arg) {
20     struct array *arr = (struct array *) arg;
21     FILE *f = fopen("test.txt", "r");
22     if (f == NULL) {
23         perror("fopen");
24         return NULL;
25     }
26     while (!feof(f)) {
27         char word[100];
28         fscanf(f, "%s", word);
29         if (word[0] != '\0') {
30             arr->frequency++;
31         }
32     }
33     fclose(f);
34     return arr;
35 }
```

Terminal Output:

```
Top 10 Words:
the 1861396
of 593677
and 416629
one 411764
in 372201
a 325874
to 316376
zero 264975
nine 258430
two 192644
The total time including child processes is: 7.758493 minutes
```

Figure 3: Multiprocessing result (6 child processes)

```
1 > #include ...
9
10 #define MAX_SIZE 1705210 // Total words in the file
11 #define UNIQUE_SIZE 255000 // Estimated unique words
12 #define NUM_PROCESSES 4 // Number of child processes
13
14 struct array {
15     char word[100];
16     int frequency;
17 };

Terminal Local x Ubuntu-20.04 x + v
Top 10 Words:
the 1061396
of 593677
and 416629
one 411764
in 372201
a 325873
to 316376
zero 264975
nine 250430
two 192644
The total time including child processes is: 10.616745 minutes
```

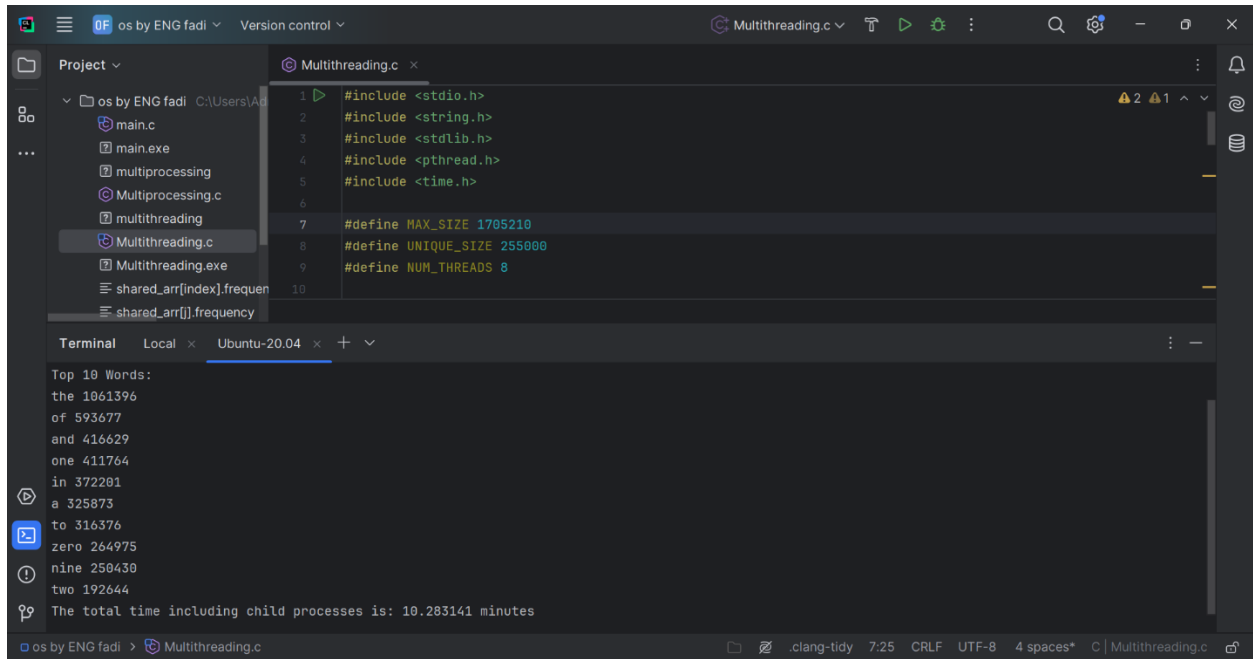
Figure 4: Multiprocessing result (4 child processes)

```
1 > #include ...
9
10 #define MAX_SIZE 1705210 // Total words in the file
11 #define UNIQUE_SIZE 255000 // Estimated unique words
12 #define NUM_PROCESSES 2 // Number of child processes
13
14 struct array {
15     char word[100];
16     int frequency;
17 };

Terminal Local x Ubuntu-20.04 x + v
Top 10 Words:
the 1061396
of 593677
and 416629
one 411764
in 372201
a 325873
to 316376
zero 264975
nine 250430
two 192644
The total time including child processes is: 15.187493 minutes
```

Figure 5: Multiprocessing result (2 child processes)

3. Multithreading Approach:



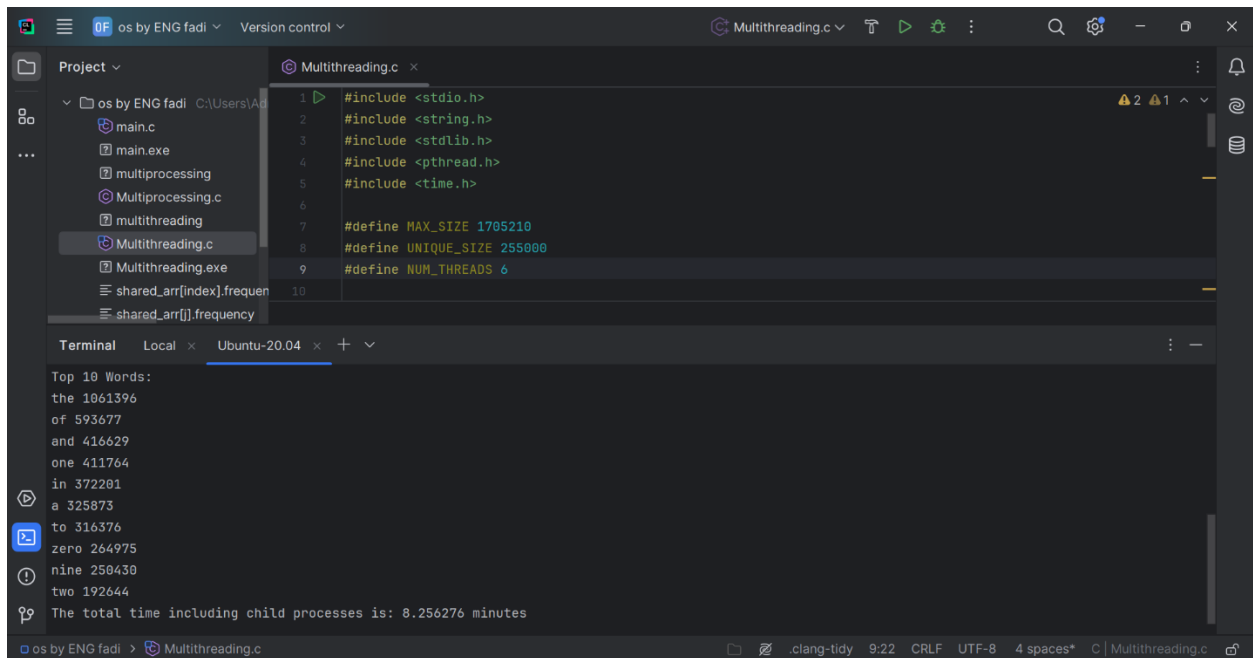
The screenshot shows the Visual Studio Code editor with the file `Multithreading.c` open. The code includes headers for `stdio.h`, `string.h`, `stdlib.h`, `pthread.h`, and `time.h`. It defines `MAX_SIZE` as 1705210, `UNIQUE_SIZE` as 255000, and `NUM_THREADS` as 8. The terminal output shows the top 10 words and the total time including child processes is 10.283141 minutes.

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <pthread.h>
#include <time.h>

#define MAX_SIZE 1705210
#define UNIQUE_SIZE 255000
#define NUM_THREADS 8
```

Top 10 Words:
the 1061396
of 593677
and 416629
one 411764
in 372201
a 325873
to 316376
zero 264975
nine 250430
two 192644
The total time including child processes is: 10.283141 minutes

Figure 6: Multithreading result (8 child threads)



The screenshot shows the Visual Studio Code editor with the file `Multithreading.c` open. The code includes headers for `stdio.h`, `string.h`, `stdlib.h`, `pthread.h`, and `time.h`. It defines `MAX_SIZE` as 1705210, `UNIQUE_SIZE` as 255000, and `NUM_THREADS` as 6. The terminal output shows the top 10 words and the total time including child processes is 8.256276 minutes.

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <pthread.h>
#include <time.h>

#define MAX_SIZE 1705210
#define UNIQUE_SIZE 255000
#define NUM_THREADS 6
```

Top 10 Words:
the 1061396
of 593677
and 416629
one 411764
in 372201
a 325873
to 316376
zero 264975
nine 250430
two 192644
The total time including child processes is: 8.256276 minutes

Figure 7 : Multithreading result (6 child threads)

The screenshot shows the Visual Studio Code interface with the file `Multithreading.c` open. The code defines `MAX_SIZE` as 1705210, `UNIQUE_SIZE` as 255000, and `NUM_THREADS` as 4. The terminal output shows the top 10 words and the total time including child processes is 9.126498 minutes.

```
#include <string.h>
#include <stdlib.h>
#include <pthread.h>
#include <time.h>

#define MAX_SIZE 1705210
#define UNIQUE_SIZE 255000
#define NUM_THREADS 4

struct array {
```

Top 10 Words:
the 1061396
of 593677
and 416629
one 411764
in 372201
a 325873
to 316376
zero 264975
nine 250430
two 192644
The total time including child processes is: 9.126498 minutes

Figure 8: Multithreading result (4 child threads)

The screenshot shows the Visual Studio Code interface with the file `Multithreading.c` open. The code defines `MAX_SIZE` as 1705210, `UNIQUE_SIZE` as 255000, and `NUM_THREADS` as 2. The terminal output shows the top 10 words and the total time including child processes is 10.255081 minutes.

```
#include <string.h>
#include <stdlib.h>
#include <pthread.h>
#include <time.h>

#define MAX_SIZE 1705210
#define UNIQUE_SIZE 255000
#define NUM_THREADS 2

struct array {
```

Top 10 Words:
the 1061396
of 593677
and 416629
one 411764
in 372201
a 325873
to 316376
zero 264975
nine 250430
two 192644
The total time including child processes is: 10.255081 minutes

Figure 9: Multithreading result (2 child threads)

1- Describe your environment: computer specs (Cores? Speed? Memory?), OS, programming language, and IDE tool, and whether you used a virtual machine.

In this project, I have used the **C programming language** to implement the solution and **Visual Studio Code** (VS Code) as my IDE. The program was executed directly on my physical machine without using a virtual machine. My system specifications include the the operating system version which is **Microsoft Windows 11 Pro**, Version **10.0.22631**, **Build 22631** and a 12th Gen Intel(R) Core (TM) i7-1255U processor with 10 physical cores, 12 logical processors, a base speed of **1.70 GHz**, and virtualization enabled. Additionally, it is supported by an L1 cache of 928 KB, an L2 cache of 6.5 MB, and an L3 cache of 12.0 MB.

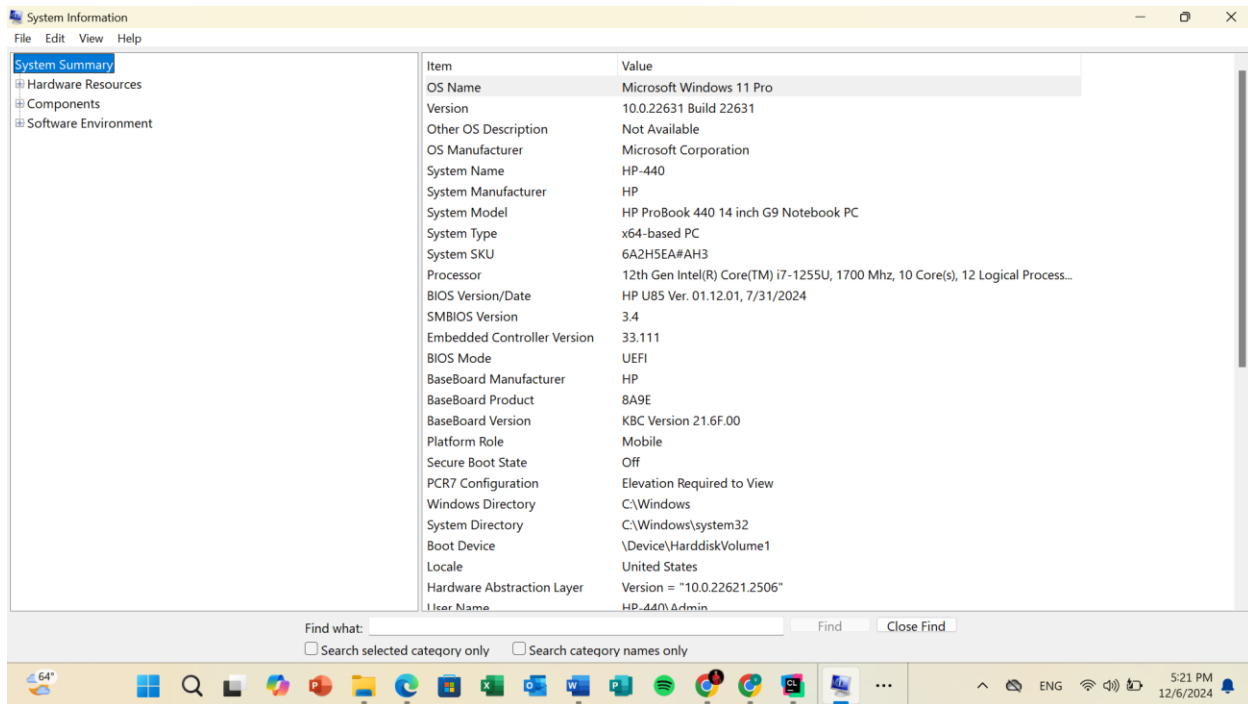


Figure 10: Operating System information

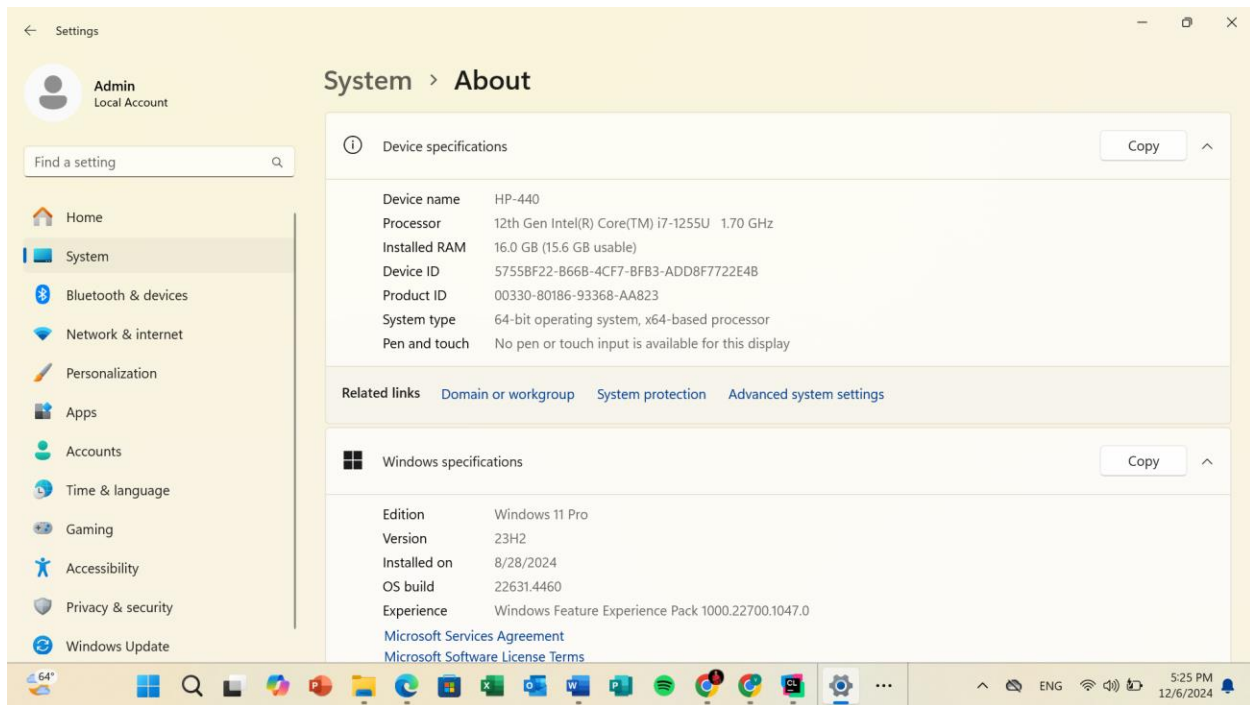


Figure 11: The Device's and window's OS specifications

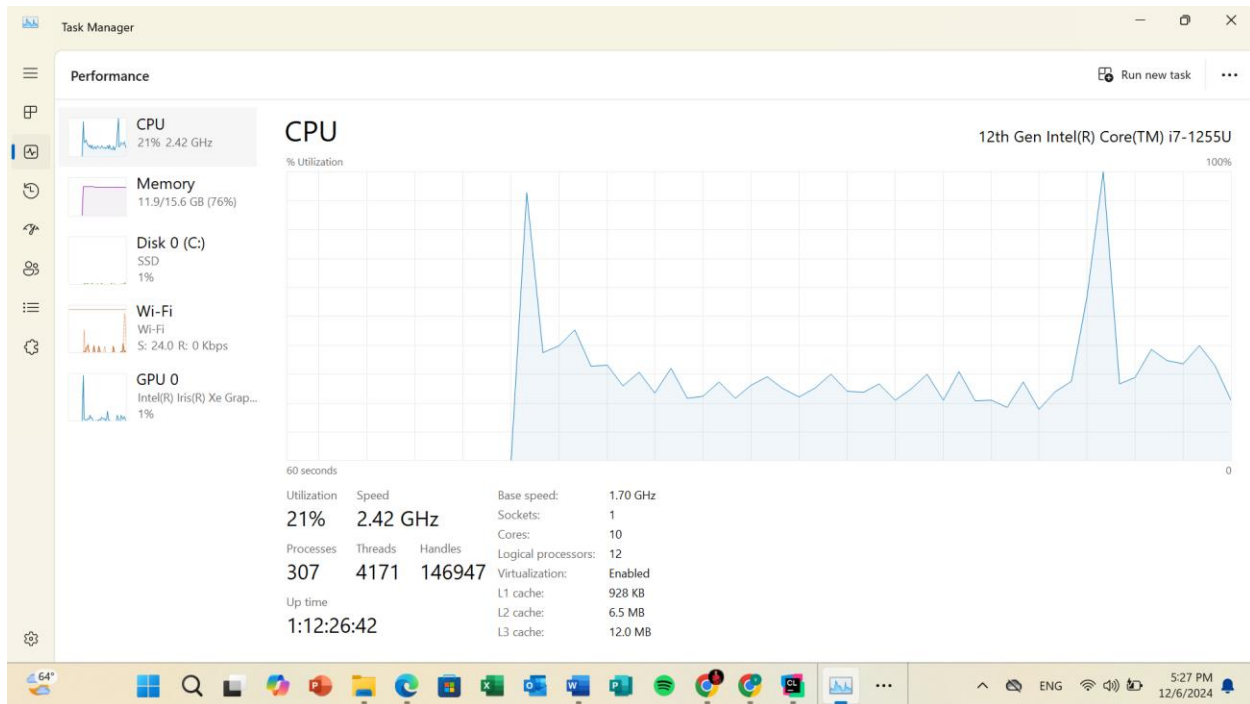


Figure 12: Processor's related details

2- How you achieved the multiprocessing and multithreading requirements, i.e. The API and functions that you used.

Multiprocessing (Using fork () and wait ())

When a program invokes the **fork ()** system call, it creates a new process by duplicating itself. This newly spawned process, called a child process, is an exact replica of the parent process at the time of creation. The **fork()** mechanism allows the parent process to generate multiple child processes, each capable of executing its tasks independently and concurrently, much like a single cell dividing into two identical cells that can function autonomously.

The **fork()** system call is used in the provided code to create multiple child processes. It allows for parallel computation of the top ten words in a text dataset. Once the child processes are created, the parent process goes into a synchronization phase to make sure all the child processes terminate before it continues the execution. The **wait()** system call causes the suspension of the parent process until each child finishes its tasks.

Multithreading (Using POSIX Threads)

Pthreads offer a threading API in the C programming language for creating multiple execution flows within one process. Now, this allows multithreading, permitting programs to achieve parallelism and distribution by splitting into smaller units of tasks that can execute in parallel. **POSIX** threads in the code snippet given are used to create several threads in one process for the parallel computation of the top ten words occurring in a text dataset.

The **pthread_create ()** function is used to create new threads, and provide a function pointer (the thread function) along with any necessary arguments. These threads run concurrently and allow for parallel processing of the dataset with great efficiency. To ensure proper synchronization and coordination, the **pthread_join ()** function is used. It guarantees that the main thread waits until all the worker threads have finished their tasks, preserving order and integrity in the program.

3- An analysis according to Amdahl's law. What percentage is the serial part of your code? What is the maximum speedup according to the available number of cores? What is the optimal number of child processes or threads?

I calculated the total time to obtain the final result and the time it takes to pass words in the bigger array containing duplicates into a smaller array with new frequencies.

I set the clock start, clock end.

```
struct array *arr1 = insertArray();
const clock_t start = clock();
struct array *arr2 = compareAndCreateArray(arr1);
const clock_t end = clock();
double cpu_time_used = ((double) (end - start)) ;
printf(format: "The parallel time is: %f minutes\n", cpu_time_used / 60);
```

Figure 13: Parallel execution time code

Time taken = 36.327685

I used this law to calculate the percentage of serial part in my code:

$T_s = \text{Total time} - T_p = 36.327685 - 36.326786$

$S = T_{\text{serial}} / T_{\text{total}} = 2.47469664 * 10^{-5}$

percentage of serial part in my code = 0.000275 %

What is the maximum speedup according to the available number of cores?

An analysis according to Amdahl's law:

$1 / (S + [(1-S) / N]) = 9.997773269$

What is the optimal number of child processes or threads?

The optimal number of child processes and threads seems to be 6. With 6 children, the execution times are consistently lower compared to having 2, 4, or 8 children processes or threads. This configuration shows significant improvement over two and four children processes or threads and doesn't introduce the higher variability seen with children processes or threads.

4- A table that compares the performance of the 3 approaches.

Program / Runtime	Runtime (minutes)
Naive	36.327685
Multiprocessing (2 Chlds)	15.187493
Multiprocessing (4 Chlds)	10.616745
Multiprocessing (6 Chlds)	7.758493
Multiprocessing (8 Chlds)	8.338094
Multithreading (2 Chlds)	10.255081
Multithreading (4 Chlds)	9.126498
Multithreading (6 Chlds)	8.256276
Multithreading (8 Chlds)	10.283141

Table 1: Comparison of the execution times of the three approaches

5- Comment on the differences in performance

From the table, I can make several observations regarding the performance comparisons between different approaches:

1. Naive Performance:

The native approach (likely **single-threaded** or **non-parallelized**) has the lowest execution time compared to multiprocessing and multithreading, which indicates the overhead involved in them.

2. Multiprocessing performance:

The execution time decreases significantly with 6 child processes compared to 2 and 4 child processes. After 6 child processes, additional children generally cause the execution time to remain stable or increase slightly. But, in our case the execution time seemed to be increased.

3. Multithreading performance:

While multithreading doesn't not outperform multiprocessing in all cases, still shows good efficiency gains, especially when the thread count matches the workload. Its similar to multiprocessing as execution time increases as we split the workload on more threads. But, when having more than 6 threads the execution time increases. Its shared memory model may introduce some minor overheads compared to multiprocessing but reduces communication between threads.

Overall, both techniques outperform the naive approach, with multiprocessing having a slight edge in performance especially with the optimal number of child processes.

Conclusion:

This project compares three ways of determining the top ten frequent words in a dataset: single-threaded naive approach, multiprocessing, and multithreading. It compares their efficiency on a system that has at least 4 cores, looks into scalability through Amdahl's law, and determines the best number of processes/threads experimentally. The multiprocessing solution makes use of **fork ()** and **wait ()** for creating independent processes, while the multithreading solution utilizes POSIX Threads for the concurrent execution of tasks. Both parallel methods show significant performance gains over the naive approach, but careful consideration of process/thread count is crucial. Overall, leveraging parallel computing enhances efficiency in the top ten frequent words computation tasks.